

Arguments, Syntax & Automation

In the last PDF, the user had to MANUALLY *tell the program* what you wanted. In **certain commands**, we can just type what we want it to do *alongside the command*, like below;

> **cd Downloads** ← Downloads is an argument, thus a VARIABLE

See how annoying it would be to have to go *INSIDE* the cd script and *THEN put in the path?*

What we can do, is actually change our **backup script**, to take *in arguments* **INSTEAD** of from inside the program.

```
#!/usr/bin/env bash

read -p "Do you want to backup (e)dited files, or (a)ccessed files from today?" answer

mkdir -p ~/backups 2>/dev/null # Makes the backup folder, only if it doesn't exist. If it does, continues.

if [ "$answer" = "e" ]; then
    find ~ -type f -mtime 0 -exec cp {} ~/backups \; 2>/dev/null
elif [ "$answer" = "a" ]; then
    find ~ -type f -atime 0 -exec cp {} ~/backups \; 2>/dev/null
fi

echo "Backed up files."
```

Currently it looks like this.

How accessing arguments works is **simple**. You *call the variable* with **\$1-9**, and the **1-9** corresponds, to the *next word after the command name*, SAY;

> **command_name any-argument any-argument-2**

any-argument → \$1 any-argument-2 → \$2

Just like how you accessed answer with \$answer, we can access the arguments, based on the **position they appear** after the command.

If we type, `backup_work e` OR `backup_work a`. We can call this variable via `$1`, it will correspond to that `e` or `a`.

- Now we do NOT read the `read -p`.

```
#!/usr/bin/env bash

mkdir -p ~/backups 2>/dev/null # Makes the backup folder, only if it doesn't exist. If it does, continues.

if [ "$1" = "e" ]; then
    find ~ -type f -mtime 0 -exec cp {} ~/backups \; 2>/dev/null
elif [ "$1" = "a" ]; then
    find ~ -type f -atime 0 -exec cp {} ~/backups \; 2>/dev/null
```

All we did was change answer to `$1`, and remove the `read -p`.

When it comes to arguments, you want to structure them the way LINUX does it.

- FIRSTLY, Do arguments with a "--", so `-e` or `-a`.
- SECONDLY, *allow* LONG FORM words of each argument with a "--", `--edited` or `--accessed`.
- THIRDLY, include a help argument, the standard is `-h` or `--help`.

For the first one, just change the `"e"` to `"-e"` and `"a"` to `"-a"`.

For the second one, you add `"||"`, which means OR, like follows.

```
if [ "$1" = "-e" ] || [ "$1" = "--edited" ];
```

```
elif [ "$1" = "-a" ] || [ "$1" = "--accessed" ];
```

For the third one, just add another `elif`, with an echo about `-e` and `-a`.

```
elif [ "$1" = "-h" ] || [ "$1" = "--help" ];
```

```
    echo "Do -e or --edited to backup today's edited files, or -a or --
    accessed to backup today's accessed files."
```

It should look like below now.

```
#!/usr/bin/env bash

mkdir -p ~/backups 2>/dev/null # Makes the backup folder, only if it doesn't exist. If it does, continues.

if [ "$1" = "-e" ] || [ "$1" = "--edited" ]; then
    find ~ -type f -mtime 0 -exec cp {} ~/backups \; 2>/dev/null
elif [ "$1" = "-a" ] || [ "$1" = "--accessed" ]; then
    find ~ -type f -atime 0 -exec cp {} ~/backups \; 2>/dev/null
elif [ "$1" = "-h" ] || [ "$1" = "--help" ]; then
    echo "Type -e OR --edited to backup today's edited files, and -a --accessed to backup today's accessed files"
fi

echo "Backed up files."
```

Now, we did this for ease, and MORE IMPORTANTLY; **automation**.

- **This is because**, now that we don't have to MANUALLY input; we can make the computer *automatically put arguments in*, and **schedule** that command WITH those specific arguments.

crontab -e is a command used to SCHEDULE programs to run.

- It will ask you which editor, *just put in 1*.

Scroll DOWN to the bottom.

- **crontab follows a syntax;**
- * * * * * path_to_command_to_run
- | | | | |
- | | | | — Day of week (0-6, 0=Sunday)
- | | | — Month (1-12)
- | | — Day of month (1-31)
- | — Hour (0-23)
- — Minute (0-59)

```
0 17 * * * ~/scripts/backup_work -e
```

THIS, backs up **edited files** *everyday* at 5pm. Via *the path* of the **script**.

BONUS

We can actually allow **BOTH arguments**. This syntax is REALLY messy, and DOES NOT *cover all the orders of inputting the keys*, but works.

```
if ( [ "$1" = "-e" ] && [ "$2" = "-a" ] ) || ( [ "$1" = "--edited" ] && [ "$2" = "--accessed" ] ); then
```

```
    find ./ -type f -mtime 0 -exec cp {} ~/backups \; 2>/dev/null
```

```
    find ./ -type f -atime 0 -exec cp {} ~/backups \; 2>/dev/null
```

```
elif ( [ "$1" = "-e" ] && [ "$2" = "-a" ] ) || ( [ "$1" = "--edited" ] && [ "$2" = "--accessed" ] ); then
    find ~ -type f -mtime 0 -exec cp {} ~/backups \; 2>/dev/null
    find ~ -type f -atime 0 -exec cp {} ~/backups \; 2>/dev/null
```

That is TRULY, the basics of BASH scripting.